

Research Notebook

CSEG3060 • Research Methodology in Computer Science

Your Full Name

SAP ID 500XXXXXX

Batch B5, B6

Track Cloud Computing and Virtualization Technology

Email yourname@stu.upes.ac.in

B.Tech. Computer Science and Engineering • Final Year

School of Computer Science • UPES

Course instructor

Dr. Mohsin Furkh Dar

Assistant Professor • Cloud & Software Operations Cluster

Academic session August–December 2026

Compiled August 17, 2026

Declaration. This notebook records my own understanding and progress. Where I have discussed ideas with classmates or worked in a group during class activities, the entries below are still written in my own words. All sources are cited, and any assistance is disclosed in Section 13.

How to use this notebook

This notebook has **fifteen sections**, in a fixed order. They are not fifteen separate assignments — they are one research idea, developed step by step. By the end of the semester the notebook should answer:

What is my research problem, why does it matter, what has already been done, what is still missing, how will I investigate it, how will I evaluate it, and what contribution do I expect to make?

The four rules

1. **Maintain it individually.** Group discussion is encouraged; the notebook is yours.
2. **Add something after every lecture.** At least one meaningful entry, every class. Record it in the log below.
3. **Show the evolution of your thinking.** Do not overwrite. Use `\version{2}{date}{why}` to stamp a revision and keep the earlier text, or `\wasnow{old}{new}` to show a phrase changing. Changes are evidence of research, not of error.
4. **Use credible sources.** Every paper goes into `references.bib` with enough detail to find it again.

What is in each section

- A blue **WHAT THIS SECTION IS FOR** box — read it first.
- An amber **COMMON TRAP** box — the mistake most students make here.
- A grey **WORKED EXAMPLE** box — a complete example, carried through all fifteen sections so you can watch one idea develop. **Delete each grey box once you have written your own.**
- A black-framed **YOUR WORK** box — where you write.
- A teal **DONE WHEN** box — the checklist to test yourself against.

Useful commands

Command	What it does
<code>\fillme{a one-sentence question}</code>	Grey italic placeholder. Replace it.
<code>\version{2}{2026-09-03}{after I.6}</code>	Stamps a revision, so the evolution stays visible.
<code>\wasnow{old wording}{new wording}</code>	Strikes the old wording, shows the new.
<code>\rnsup{A sub-heading}</code>	A heading inside a box.
<code>\todo{find the dataset licence}</code>	An amber reminder to yourself.
<code>\citet{key} / \cite{key}</code>	Cites a paper from <code>references.bib</code> .

Lecture log

One line after every class. This is the fastest way for both of us to see that the notebook is being maintained continuously rather than assembled the night before. *(If you would rather not keep a log, delete this subsection — nothing else depends on it.)*

Date	Lecture	What I added or changed
[2026-08-04]	L0	[listed three areas that interest me]

Contents

How to use this notebook	2
1 Research Interest	5
2 Problem Ideas	7
3 Problem Statement	9
4 Research Questions	12
5 Research Objectives	14
6 Literature Papers	16
7 Literature Matrix	19
8 Research Gap	23
9 Proposed Methodology	25
10 Dataset / Data Source	27
11 Experimental Plan	29
12 Evaluation Metrics	31
13 Ethical Considerations	33
14 Research Proposal	35
15 Final Presentation	37

1 Research Interest

Lecture 1.1–1.2 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Name the areas of computer science you are actually willing to spend two semesters inside. This is the widest section of the notebook and the only one you are allowed to be vague in — everything after it narrows.

Write **three** candidate areas, not one. For each, answer in one line each: what draws you to it, what you have already built or read in it, and who outside this classroom works on it. An area you cannot name a single practitioner or paper for is a preference, not a research interest yet.

COMMON TRAP

Method-first framing. “I want to use transformers / Kubernetes — now let me find a problem.” You will force a method onto a problem that never needed it, and a reviewer will ask why. Choose the area by the *questions* it asks, not the tools it uses.

WORKED EXAMPLE — read it, then delete this whole box

Area 1: Serverless computing and function-as-a-service. Why: I built a small FaaS deployment for a hackathon and half the complaints were about the first request being slow. Already done: OpenFaaS on a 3-node k3s cluster; read the Berkeley view on serverless [Jonas et al. \[2019\]](#). Who works on this: the systems group at UW–Madison ([Fuerst and Sharma](#)), USENIX ATC and ASPLOS.

Area 2: Cluster scheduling and resource management. Why: I want to understand how a scheduler decides, not just how to write a YAML file. Already done: Kubernetes lab work; read the Borg paper [Verma et al. \[2015\]](#). Who works on this: Google, Microsoft Research ([Cortez et al.](#)), EuroSys and SOSP.

Area 3: Energy and carbon-aware cloud operation. Why: campus servers run hot and idle at the same time, which seems absurd. Already done: nothing yet beyond reading. Who works on this: HotCarbon workshop, Microsoft Research.

Leading choice, for now: Area 1. It is the only one where I already have a running system I can measure tonight, which makes the feasibility row of FINER survivable.

YOUR WORK

Area 1: *[name the area]*

Why it interests me. *[2–3 sentences, specific]*

What I have already done here. *[a course, a mini-project, a paper you read]*

Who works on this. *[one lab, company, or author, and one venue]*

Area 2: *[name the area]*

Why it interests me. *[2–3 sentences]*

What I have already done here. *[...]*

Who works on this. *[...]*

Area 3: *[name the area]*

Why it interests me. *[2–3 sentences]*

What I have already done here. [...]

Who works on this. [...]

My leading choice, for now

[which of the three, and the single reason]

This may change. If it does, say so here rather than deleting the old text.

DONE WHEN

Three areas named. Each has a named venue, lab or author attached. One is marked as the current front-runner, with a reason.

2 Problem Ideas

Lecture I.2–I.3 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Turn the leading area into **five candidate problems**. Problem-finding is a search, not a spark — run the five-source sweep from Lecture I.2 and record which source produced each idea:

- S1. Literature gap** — explicit (Future Work), population, contradiction, or assumption.
- S2. Professional practice** — your own measurable irritation.
- S3. Replication** — a published result nobody has checked.
- S4. Interdisciplinary overlap** — CS $\times$ another field.
- S5. Industry pain** — issue trackers, post-mortems, engineering blogs.

Ideas sitting in *two* sources are usually strongest: one source supplies the novelty, the other supplies the significance.

COMMON TRAP

“No papers exist, so this is a good topic.” Sometimes nobody asked because the answer is obvious or useless. Absence of recent literature usually means *closed*, not open. Check before you celebrate.

WORKED EXAMPLE — read it, then delete this whole box

#	Candidate idea	Source	Why it might matter	Keep?
1	Cold-start latency under memory pressure	S2 + S1	My own cluster shows it; published policies assume plentiful RAM	Y
2	Reproduce FaaSCache’s reported hit-ratio gains on a different trace	S3	Success criteria already published; either outcome is a result	Y
3	Carbon-aware placement of batch jobs	S1 + S4	Real gap, but I have no power meter and no access to grid-intensity data	N
4	Autoscaler tuning for bursty student-project workloads	S2	Measurable pain on our own cluster, but very close to idea 1	Y
5	Live-migration overhead for stateful containers	S1	Interesting, but the setup alone would take a semester	N

Taking forward: (1) keep-alive policy under memory pressure; (2) replication of FaaSCache-style eviction on the Azure trace; (4) autoscaler behaviour under bursty load.

Rejected: idea 3 — no access to per-node power measurement or regional carbon-intensity data inside two weeks, so feasibility scores 1. Idea 5 — setup cost exceeds the whole time budget.

YOUR WORK

Record every idea you generate, including the ones you reject — rejected ideas with a stated reason are evidence of judgement, and they are marked.

#	Candidate idea	Source	Why it might matter	Keep?
1	[...]	S[...]	[...]	[Y/N]
2	[...]	S[...]	[...]	[Y/N]
3	[...]	S[...]	[...]	[Y/N]
4	[...]	S[...]	[...]	[Y/N]
5	[...]	S[...]	[...]	[Y/N]

The three I am taking forward

1. *[idea, one sentence]*
2. *[idea, one sentence]*
3. *[idea, one sentence]*

Rejected, and why

[one line per rejected idea — “needs hospital data I cannot get”, “already solved in 2019”, “I could not name a metric”]

DONE WHEN

Five ideas recorded with their source number. At least one comes from source 2 (your own practice). Three carried forward, and every rejection has a stated reason.

3 Problem Statement

Lecture 1.3–1.5 Update this section after these lectures.

WHAT THIS SECTION IS FOR

One problem, narrowed until roughly **one** final-year project would satisfy the sentence. Use the funnel — *interest* → *field* → *topic* → *task* → *problem* → *question* — and turn about three of the five knobs: **population, task, comparison, metric, setting**.

Then write it in one of the five templates:

- **T1 Comparative** — “Does X improve *metric* over *baseline*, on *data*, under *constraint*?” (the default)
- **T2 Transfer** — “To what extent does a method validated on P_1 retain *metric* on P_2 ?”
- **T3 Explanatory** — “Why does *method* fail on *cases*, and which factor accounts for it?”
- **T4 Feasibility** — “Can a system achieve *outcome* subject to C_1 and C_2 , and at what cost?”
- **T5 Measurement** — “Does metric M actually correlate with outcome O that we claim to care about?”

The full statement is **six moves in under 200 words**: context, gap, objective, question, hypothesis, scope. Then score it on **FINER + M** — the criteria *multiply*, so any single 1 is fatal.

COMMON TRAP

Skipping the scope line. Scope says what you *will not* do. It is the only thing that stops a supervisor, an examiner, or you at 2 a.m. from quietly expanding the project.

WORKED EXAMPLE — read it, then delete this whole box

Narrowing trace. Interest: cloud computing → Field: serverless → Topic: cold starts → Task: keep-alive / eviction policy → Problem: published policies are evaluated with memory to spare, but our cluster does not have memory to spare.

Knobs turned: *setting* (memory-constrained, 8 GB/node rather than a large cloud), *population* (ML-inference functions with large container images, not **hello-world**), *metric* (p95 cold-start latency and cost per kept-warm GB-second, not mean latency alone). **Projects that fit:** about one.

[Context] Serverless platforms bill per invocation and reclaim idle containers, so the first request after an idle period pays a cold start — hundreds of milliseconds to several seconds for ML-inference functions with large images. Production traces show most functions are invoked rarely, so cold starts dominate tail latency [Shahrad et al. \[2020\]](#). **[Gap]** Keep-alive and caching policies such as FaaS-Cache [Fuerst and Sharma \[2021\]](#) are evaluated where memory is ample relative to the working set; their behaviour when total keep-alive memory is a small fraction of it is reported far less often, and the mechanism of degradation is not established. **[Objective]** This work measures and explains that degradation on a memory-constrained cluster. **[Question]** Does a greedy-dual keep-alive policy retain its cold-start reduction over a fixed 10-minute keep-alive baseline, measured as p95 end-to-end latency, when total keep-alive memory is under 25% of the function working-set size, replaying the Azure Functions trace on a 3-node 8 GB k3s cluster? **[Hypothesis]** It will not; the advantage will fall below 20% of the reported gain, and eviction thrashing on large-image functions will account for more of the loss than request arrival burstiness. **[Scope]** Three function images

only; no GPU functions; no cross-region placement; no cost modelling beyond GB-seconds.

FINER + M (self-scored). F4 — cluster and trace both in hand, pilot ran in week 1. I4 — platform engineers running small on-prem FaaS; also my own lab. N4 — type 3, new context: known policy, untested memory regime. E5 — no human data; trace is public and permissively licensed. R4 — if true, small-cluster operators should not adopt the policy unmodified. M4 — p95 latency and GB-seconds, both instrumented. **Lowest row: N and F, both 4.** No row scores 1.

Repair applied: re-anchor. The first version compared against “no keep-alive”, which is not what anyone actually runs. The baseline is now fixed 10-minute keep-alive, i.e. current default practice.

YOUR WORK

Version 1 • [date] • initial statement, after Lecture 1.3

Narrowing trace

Interest [...] → **Field** [...] → **Topic** [...] → **Task** [...] → **Problem** [...]

Knobs I turned: [population / task / comparison / metric / setting — name about three]

How many final-year projects fit my sentence? [aim for about 1]

Problem statement (six moves, under 200 words)

[Context] [why the area matters — 2 lines]

[Gap] [what is not known — 1 line]

[Objective] [what you will do about it]

[Question] [one sentence, in template T__]

[Hypothesis] [what you expect, stated so it could be wrong]

[Scope] [three things you will NOT do]

FINER + M scorecard

Criterion	Score (1–5)	Evidence — a file, a paper, or a named person
Feasible	[...]	[...]
Interesting	[...]	[name a specific group who would act differently]
Novel	[...]	[which of the four novelty types, and three papers you stand next to]
Ethical	[...]	[...]
Relevant	[...]	[run the “so what” chain until someone acts differently]
Measurable	[...]	[construct → operational definition → metric → threshold]

Lowest row = my project’s real risk [...]

Repair applied

[narrow / re-anchor / re-measure / abandon — and what changed]

Revisions (append below, never overwrite)

Version 2 • [date] • [what changed and why]

[revised statement]

DONE WHEN

About one project fits the sentence. All six moves present, under 200 words, in a named template. Every FINER + M row has a score *and* evidence. No row scores 1. The scope line names three exclusions.

4 Research Questions

Lecture 1.6 Update this section after these lectures.

WHAT THIS SECTION IS FOR

One **main research question** (the problem statement's question move, restated cleanly) and **two to four sub-questions**, each answerable by a single experiment or analysis. Questions are interrogative and neutral. Hypotheses are declarative and take a risk. Directional hypotheses ("will drop by more than 10 points") are stronger than non-directional ones ("will differ"), because they can be wrong in a specific way. Commit to what counts as support *before* you run anything.

COMMON TRAP

Questions that cannot be wrong. "How can we improve system performance using modern techniques?" No result could refute it, so no result can support it either. If you cannot state the finding that would make you abandon the hypothesis, it is not a research question.

WORKED EXAMPLE — read it, then delete this whole box

RQ0. Does a greedy-dual keep-alive policy retain its cold-start advantage over fixed 10-minute keep-alive when keep-alive memory is under 25% of the function working-set size?

ID	Sub-question	Answered by
RQ1	How does the p95 cold-start latency gap between the two policies vary as the memory ratio falls from 100% to 10%?	E1, memory sweep
RQ2	Which factor accounts for more of the loss — eviction thrashing on large images, or arrival burstiness?	E2, ablation
RQ3	Does the policy's advantage differ between small (<150 MB) and large (>1 GB) images?	E3, stratified analysis

H1. Below a 25% memory ratio the p95 advantage falls under 20% of the reported gain. *Refuted if* the advantage stays above 50% of the reported gain across all ratios down to 10%.

H2. Image size explains more variance in the latency gap than inter-arrival burstiness (CV of inter-arrival time). *Refuted if* burstiness has the larger standardised coefficient.

YOUR WORK

Main research question

RQ0. *[one sentence, the same question as in Section 3]*

Sub-questions

ID	Sub-question	Answered by
RQ1	<i>[...]</i>	<i>[which experiment]</i>
RQ2	<i>[...]</i>	<i>[...]</i>
RQ3	<i>[...]</i>	<i>[...]</i>

Hypotheses

ID	Hypothesis (directional)	Result that would refute it
H1	[...]	[...]
H2	[...]	[...]

Revisions

Version 2 • [date] • [what changed after which lecture]

DONE WHEN

One main question, two to four sub-questions, each mapped to one experiment. Every hypothesis is directional and has a stated refuting result.

5 Research Objectives

Lecture 1.6 Update this section after these lectures.

WHAT THIS SECTION IS FOR

One **general objective** (the research question restated as an intention) and **three to five specific objectives** that, if all completed, discharge it.

Use measurable verbs — *measure, compare, quantify, characterise, evaluate, attribute, implement*. Avoid *understand, explore, study, look into*: they name no observable, so they cannot be checked off.

Each specific objective must map to exactly one experiment and produce exactly one artefact — a table, a figure, or a released component. **An objective that produces no artefact is a wish.**

COMMON TRAP

Objectives that are activities. “Study the literature”, “learn PyTorch”, “set up the cluster” are *tasks* on your plan, not objectives of the research. Also: more objectives than you have weeks.

WORKED EXAMPLE — read it, then delete this whole box

General objective. To measure and explain the degradation of greedy-dual keep-alive performance, relative to fixed keep-alive, as available keep-alive memory falls below the function working-set size on a constrained on-premise serverless cluster.

ID	Specific objective	Artefact	Answers
SO1	<i>Implement</i> both keep-alive policies as a pluggable controller on OpenFaaS	Released repo	—
SO2	<i>Reproduce</i> the published cold-start reduction at an unconstrained memory ratio, within 10%	Table 1	sanity check
SO3	<i>Quantify</i> p95 latency for both policies across six memory ratios, 3 seeds	Figure 2	RQ1
SO4	<i>Attribute</i> the degradation across image size and arrival burstiness by regression	Table 3	RQ2, RQ3

Weakest link: *Method* → *Data*. SO2 depends on the published setup being reproducible from the paper alone; if it is not, the whole comparison loses its anchor. Mitigation is in the fallback plan, Section 10.

In scope. Three OpenFaaS function images; the Azure Functions 2019 invocation trace; a 3-node k3s cluster. **Delimitations.** No GPU functions (different memory model, and I have one GPU). No multi-region placement. No monetary cost model. **Limitations.** Single hardware configuration, so external validity to public clouds is unestablished; the trace is one provider’s, from 2019.

YOUR WORK

General objective

[one sentence, beginning “To measure / quantify / characterise ...”]

Specific objectives

ID	Specific objective (measurable verb first)	Artefact produced	Answers
SO1	[...]	[Table / Fig.]	RQ/[...]
SO2	[...]	[...]	RQ/[...]
SO3	[...]	[...]	RQ/[...]
SO4	[...]	[...]	RQ/[...]

Chain check

Trace it left to right and name the missing link, if any:

Problem → Question → Objective → Hypothesis → Method → Data → Metric → Claim

Weakest link in my chain: [...]

Scope and delimitations

In scope. [population, tasks, datasets, platforms, time window]

Delimitations (chosen exclusions, with reasons). [...]

Limitations (imposed on me, not chosen). [...]

DONE WHEN

One general objective. Three to five specific objectives, each starting with a measurable verb, each mapped to one artefact and one RQ. Scope, delimitations and limitations are three separate lists.

6 Literature Papers

Lecture III.1–III.3 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Between **10 and 15 papers** you have actually read — not a search-result dump. For each, record enough that you never have to open it again to cite it or argue with it.

Prioritise in this order: (1) the newest survey in your area, (2) the two or three papers your work directly compares against, (3) the papers that established the datasets or metrics you will use, (4) anything that contradicts your hypothesis. **That last category is the one students skip and examiners ask about.**

Add every paper to **references.bib** as you go. Read at minimum the abstract, the last two paragraphs, and the threats-to-validity section.

COMMON TRAP

Citing what you have not read. If you cannot state a paper’s baseline, dataset and headline number in one line, you have not read it — you have seen its title. Examiners open one citation at random.

WORKED EXAMPLE — read it, then delete this whole box

P1 \cite{shahrad2020wild}

Full citation	Shahrad et al., “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider”, USENIX ATC 2020.
Problem	Real FaaS workloads were undocumented, so keep-alive policies were tuned against guesses.
Method	Characterisation of a two-week Azure Functions trace; a hybrid histogram policy derived from it.
Dataset	Azure Functions invocation trace (released publicly with the paper).
Metric	Cold-start rate and wasted memory time.
Limitation	“ <i>our evaluation assumes memory is provisioned generously relative to the working set</i> ” — exactly my setting.
Why it matters	Supplies my trace <i>and</i> the population gap my problem sits in.
Date read	2026-08-09

P2 \cite{fuerst2021faascache}

Full citation	Fuerst and Sharma, “FaasCache: Keeping Serverless Computing Alive with Greedy-Dual Caching”, ASPLOS 2021.
Problem	Keep-alive is a caching problem, but was being treated as a fixed timeout.
Method	Greedy-dual eviction over containers, with a simulator and an OpenWhisk implementation.
Dataset	Azure trace, replayed.
Metric	Cold starts reduced by roughly $3\times$ vs. fixed keep-alive.
Limitation	Memory ratios explored are generous; behaviour under severe constraint is not characterised.
Why it matters	This is my treatment condition and the number I try to reproduce.
Date read	2026-08-10

(P3–P12 continue in the same form: Wang et al. [2018], Ustiugov et al. [2021], Agache et al. [2020], Hellerstein et al. [2019], Cortez et al. [2017], Verma et al. [2015]...)

Search record. Google Scholar, 2026-08-09, serverless "cold start" keep-alive memory — 214 hits, 9 kept. arXiv cs.DC, same day, serverless cold start survey 2023..2026 — 61 hits, 3 kept. **Inclusion:** peer-reviewed or arXiv, 2018 onward, reports a latency or cold-start metric. **Exclusion:** no empirical evaluation; edge-only deployments.

YOUR WORK

For each paper, fill one block. Duplicate the block as many times as needed.

P1 [cite key, e.g. \cite{smith2024}]

Full citation [authors, title, venue, year]
Problem addressed [one line]
Method [one line]
Dataset / system [...]
Metric + headline number [...]
Stated limitation [quote it, do not paraphrase]
Why it matters to me [baseline / dataset / metric / contradicts me]
Date read [...]

P2 [cite key]

Full citation [...]
Problem addressed [...]
Method [...]
Dataset / system [...]
Metric + headline number [...]
Stated limitation [...]
Why it matters to me [...]
Date read [...]

[TODO: duplicate the block above for P3 ... P12]

Search record — so this is reproducible

Database	Date	Query string	Hits / kept
[Scholar]	[...]	[...]	[...]
[arXiv]	[...]	[...]	[...]
[IEEE / ACM DL]	[...]	[...]	[...]
Inclusion criteria. [...] Exclusion criteria. [...]			

DONE WHEN

10–15 papers, each with method, dataset, metric, headline number and a quoted limitation. All are in **references.bib** and compile. At least one paper disagrees with your hypothesis. The search record lets someone repeat your search.

7 Literature Matrix

Lecture III.2–III.4 Update this section after these lectures.

WHAT THIS SECTION IS FOR

The matrix is where a reading list becomes an argument. Organise **by concept, not by paper** — a matrix whose rows are papers and whose columns are “summary” is a list in disguise.

Read the table *down the columns*. The empty cells and the columns where every row says the same thing are your research gap; Section 8 simply writes up what this table already shows.

The table is set landscape so the columns fit. Add rows freely — it breaks across pages.

COMMON TRAP

A matrix with no empty cells. If every cell is filled and every paper agrees, either your columns are too coarse or you have only read papers that support you. Add the column that distinguishes your work, and one paper that contradicts you.

Worked example — delete once you have built your own

Read down the columns. The bold cells in the last row are the ones no published row fills — that is the research gap, and Section 8 simply writes it up.

Paper	Year / venue	Problem addressed	Policy / method	Workload scale	&	Memory reported	ratio	Metric reported	Stated limitation	Relevance to me
Shahrad et al. [2020]	2020 ATC	FaaS workloads undocumented	Hybrid histogram keep-alive	Azure trace, provider scale		not reported		Cold-start rate, wasted mem.	Assumes generous memory	Supplies my trace; states the assumption
Fuerst and Sharma [2021]	2021 ASPLOS	Keep-alive treated as timeout, not caching	Greedy-dual eviction	Azure trace, Open-Whisk + sim		>100%		Cold starts (~3times fewer)	Constrained regime not characterised	My treatment condition
Wang et al. [2018]	2018 ATC	Platform internals opaque	Black-box measurement	AWS/Azure/Google, micro		n/a		Cold-start latency, isolation	Providers have since changed	Measurement methodology
Ustiugov et al. [2021]	2021 ASPLOS	Snapshot restore still slow	REAP record-and-prefetch	vHive, own harness		not reported		Restore latency	Different mechanism than keep-alive	Must hold image tech fixed
Agache et al. [2020]	2020 NSDI	VM isolation too heavy for FaaS	microVM (Firecracker)	AWS Lambda scale		n/a		Boot time, density	Not a policy paper	Explains the floor on cold start
Cortez et al. [2017]	2017 SOSP	VM demand unpredictable	Prediction service	Azure IaaS		n/a		Prediction accuracy	IaaS, not FaaS	Prediction as an alternative lever
Hellerstein et al. [2019]	2019 CIDR	Serverless limits for data work	Position paper	—		n/a		—	No empirical evaluation	Framing and counterargument
This notebook	2026	Policy behaviour under memory scarcity	Greedy-dual vs. fixed, replicated	Azure trace, 3-node 8 GB k3s		100–10%, swept		p95 e2e latency + GB-s	One hardware config	The empty column, filled

Your literature matrix

One row per paper from Section 6. Rename the columns to whatever distinguishes work in your area — the columns are the argument. Keep the last row for your own proposed work. Add rows freely; the table breaks across pages.

Paper	Year / venue	[column 1]	[column 2]	[column 3]	[column 4]	[column 5]	[column 6]	Relevance to me

Paper	Year / venue	[column 1]	[column 2]	[column 3]	[column 4]	[column 5]	[column 6]	Relevance to me
This notebook	[...]	[...]	[...]	[...]	[...]	[...]	[...]	[...]

DONE WHEN

Every paper from Section 6 appears as a row. At least one column is populated for your own proposed work, in the last row. You can point at the specific cells — empty, or uniformly identical — that justify the gap in Section 8.

8 Research Gap

Lecture III.4 Update this section after these lectures.

WHAT THIS SECTION IS FOR

A gap is not “nobody has done exactly this”. Name which **kind** of gap it is, and point at the cells in the Section 7 matrix that prove it:

- **Explicit** — the authors state it in Future Work or Limitations. Easiest, and the most crowded.
- **Population** — the method works, but was never tested on your users, language, hardware or region. Best value for a final-year project.
- **Contradiction** — paper A says X helps, paper B says it hurts, nobody reconciled them.
- **Assumption** — everyone assumes *Y* and nobody tested *Y*. Hardest to see, highest payoff.
- **Methodological** — the claim exists but the evidence for it is weak: no baseline, one seed, a metric that does not measure the construct.

COMMON TRAP

The crowded gap. You found it in a famous paper’s Future Work — so did four hundred other people, and the authors started a year ago. Combine two sources instead: their gap *plus* your population, your constraint, or your hardware.

WORKED EXAMPLE — read it, then delete this whole box

Statement. Keep-alive is now well understood as a caching problem, and greedy-dual eviction is the strongest published policy for it [Fuerst and Sharma \[2021\]](#). Every evaluation I found, however, provisions keep-alive memory generously relative to the function working set — the regime a large provider operates in. Small on-premise and edge clusters operate in the opposite regime, where keep-alive memory is a small fraction of the working set. Whether the policy’s advantage survives there, and what mechanism erodes it, is not established.

Field	Answer
Type of gap	Population , with a methodological component — the memory ratio is rarely reported at all.
Evidence from the matrix	The <i>memory ratio</i> column is empty or “>100%” for every row; no row reports p95 under 25%.
Closest papers	Fuerst and Sharma [2021] — same policy, generous memory. Shahrad et al. [2020] — same trace, different policy, and states the assumption explicitly.
Why still open	Providers do not have this constraint, so it is not their problem; small-cluster operators do not publish.
What changes	Practice. If the advantage disappears, small-cluster operators should keep the simpler fixed policy and spend the effort on image size instead.

Did the gap change my question? Yes. Reading [Ustiugov et al. \[2021\]](#) showed that snapshot restore attacks the same latency from a different direction, so a fair claim must hold image technology fixed. The scope line now says so explicitly.

YOUR WORK**Statement of the gap**

[3–5 sentences. What is known, what is not known, and why the missing part matters.]

Gap classification

Field	Your answer
Type of gap	<i>[explicit / population / contradiction / assumption / methodological]</i>
Evidence from the matrix	<i>[name the columns and rows — “no row in the X column”]</i>
Papers that come closest	<i>[2–3 cite keys, and what each still leaves open]</i>
Why nobody has closed it yet	<i>[cost? access? nobody asked? recently became possible?]</i>
What changes once it is closed	<i>[knowledge / practice / policy / cost]</i>

Did the gap change my question?

[If the literature answered your original question, say so and state the revised question. This is a good outcome, not a failure.]

Version 3 • [date] • [question revised after literature study]

DONE WHEN

The gap is one of the five named types. It is justified by specific cells in the Section 7 matrix, not by assertion. The two closest papers are named, with what each leaves open. Any change to the research question is recorded, not hidden.

9 Proposed Methodology

Lecture I.7, II.1–II.6 Update this section after these lectures.

WHAT THIS SECTION IS FOR

The method follows from the *kind of claim* you want to defend, not from the technique you already know. Pick your primary approach from Lecture I.7 and say why the other three are wrong for this question:

- **Analytical / theoretical** — evidence is a proof or a bound. Holds for all inputs; assumptions carry the argument.
- **Experimental / empirical** — evidence is measured behaviour under controlled variation. Needs a named baseline, held-out data, ≥ 3 seeds.
- **Simulation-based** — evidence is behaviour of a model, when the real system is inaccessible or too large. Validity is inherited from the simulator.
- **Design science / constructive** — evidence is an artefact plus a rigorous evaluation of it. *Problem* $\rightarrow$ *design* $\rightarrow$ *demonstration* $\rightarrow$ *evaluation*.

Most good computing work is a hybrid. Say which parts are which.

COMMON TRAP

Demonstration masquerading as evaluation. Showing that your system runs is not evidence that it is better. If the evaluation cannot come out against you, it is a demo.

WORKED EXAMPLE — read it, then delete this whole box

Primary approach: experimental. **Secondary:** design science — the pluggable policy controller is a built artefact (SO1), but the claim rests on the measurements, not on the artefact existing.

Why not the others. *Analytical:* greedy-dual has a known competitive ratio, but the question is about behaviour under a real arrival process and real image-pull costs, which the model abstracts away. *Simulation alone:* the published simulator does not model image-layer pull time, which I expect to be the dominant term under constraint — I use it only to pre-screen memory ratios before running them for real.

Pipeline. Azure trace $\rightarrow$ replay driver (scaled to cluster capacity) $\rightarrow$ OpenFaaS gateway $\rightarrow$ pluggable keep-alive controller (fixed | greedy-dual) $\rightarrow$ three function images $\rightarrow$ per-invocation latency and memory telemetry $\rightarrow$ analysis notebook.

Baselines. (i) *No keep-alive* — trivial floor, establishes the worst case. (ii) *Fixed 10-minute keep-alive* — current default practice and the real comparison. (iii) *Published greedy-dual Fuerst and Sharma [2021]* at generous memory — the number to reproduce.

Validity. *Internal:* policy is the only variable changed; cluster is drained and images pre-pulled identically between runs. *External:* one hardware configuration only — stated as a limitation, not claimed away. *Construct:* p95 end-to-end latency includes queueing, which is what a user experiences; container-start time alone would not be. *Conclusion:* 3 seeds, bootstrap CIs on p95, effect size reported alongside any test.

YOUR WORK

Approach

Primary approach. *[analytical / experimental / simulation / design science]*

Secondary approach, if hybrid. *[...]*

Why not the others. *[one line each — this is the part examiners probe]*

Pipeline

[Describe the pipeline stage by stage. Then draw it — a figure here is worth a page of prose. Use `\includegraphics` or a TikZ diagram.]

*Replace this box with your pipeline diagram
(figures/pipeline.pdf or a TikZ picture)*

Baselines — what you are comparing against

Baseline	Why this is the right comparison	Source / cite key
<i>[trivial baseline]</i>	<i>[establishes the floor]</i>	<i>[...]</i>
<i>[standard practice]</i>	<i>[what people do today]</i>	<i>[...]</i>
<i>[published SOTA]</i>	<i>[the number to beat]</i>	<i>[...]</i>

Validity

Threat	How I address it
Internal	<i>[did the thing you changed cause the effect?]</i>
External	<i>[does it hold outside your sample?]</i>
Construct	<i>[did you measure the thing you named?]</i>
Conclusion	<i>[do the statistics support the strength of the claim?]</i>

DONE WHEN

Primary approach named, with one line on why each of the other three is wrong here. A pipeline figure exists. At least two baselines, one of them trivial. All four validity threats have a stated mitigation.

10 Dataset / Data Source

Lecture 1.8 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Name the exact data, systems, users or experimental apparatus you need — and prove you can get it. **Confirm access in writing in week one.** A beautiful problem whose data sits behind an eight-month approval is not a problem you can do, and narrowing will not fix it: a smaller slice of inaccessible data is still inaccessible.

Record licence and provenance for every source. “Publicly visible” is not “public”, and “public” is not “yours to use”.

COMMON TRAP

Data leakage, designed in on day one. Separate your held-out test set *before* any preprocessing, tuning or modelling, and touch it once. Watch for preprocessing fitted on the full set, temporal shuffling of time-ordered data, and near-duplicates across splits.

WORKED EXAMPLE — read it, then delete this whole box

Source	What it gives / size	Licence	Access confirmed
Azure Functions trace 2019	14 days of invocation timestamps, function IDs, durations, memory — ~1.9 GB	CC-BY-4.0, released with Shahrad et al. [2020]	2026-08-08, downloaded
Lab k3s cluster	3 nodes × 8GB, exclusive booking	Institutional	2026-08-08, booked in writing, Tue/Thu 18:00–23:00
Function images	ResNet-50 (1.1 GB), spaCy NER (420 MB), JSON transform (95 MB)	Apache-2.0 / MIT	Built locally

Not building data myself — the trace is public. What I generate is *telemetry*, and the collection procedure is scripted: `make replay RATIO=0.25 POLICY=gd SEED=1`.

“**Splits**” here are workload segments, not ML splits: days 1–2 are the pilot and tuning segment, days 3–12 are the measurement segment, days 13–14 are held out and replayed exactly once at the end to confirm nothing was tuned to the measurement window.

Leakage audit. Policy parameters are fixed on days 1–2 only. Function-to-image assignment is fixed before any run. The held-out segment is replayed once, after all numbers are frozen.

Fallback. If the cluster booking is lost, the same experiment runs on three 8GB cloud VMs from the department’s credit; the cost is that network and disk characteristics change, which I would report as a confounder rather than ignore.

YOUR WORK

Sources

Source	What it gives me / size	Licence	Access confirmed?
[...]	[...]	[...]	[date + how]
[...]	[...]	[...]	[...]
[...]	[...]	[...]	[...]

If I have to build the data myself

Collection procedure. *[detailed enough that a stranger could repeat it]*

Sampling method. *[random / stratified / purposive / convenience — and declare convenience honestly]*

Target size, and why that size. *[...]*

Annotation. *[who labels, against what guideline, how agreement is measured]*

Splits

Split	Size	How separated (and when)
Train	[...]	[...]
Validation	[...]	[...]
Test (touched once)	[...]	<i>[separated before any preprocessing]</i>

Leakage audit

[Preprocessing fitted on train only? Time order preserved? Duplicates across splits checked? Test set used exactly once?]

Fallback plan

[If the primary source falls through, what do I use instead — and what does that cost the claim?]

DONE WHEN

Every source has a licence and a confirmed access route with a date. Splits are defined, with the test set separated before preprocessing. A leakage audit is written. A fallback source exists.

11 Experimental Plan

Lecture 1.8, 11.7 Update this section after these lectures.

WHAT THIS SECTION IS FOR

The experiment that would settle the question, described so precisely that someone else could run it and get your numbers.

State independent variables (what you change), dependent variables (what you measure), and controlled variables (what you hold fixed). Run **at least three seeds** — a single run is an anecdote — and report mean with standard deviation.

Then **cost it before you commit**. Scope creep is multiplicative, not additive: one extra baseline multiplies through every dataset, seed and ablation.

COMMON TRAP

An unfair tuning budget. Tuning your method for a week and running the baseline once, out of the box, is not a comparison. Give the baseline the same budget, and say what budget that was.

WORKED EXAMPLE — read it, then delete this whole box

Variables. *Independent:* keep-alive policy (fixed | greedy-dual), memory ratio (100, 75, 50, 25, 15, 10%). *Dependent:* p95 end-to-end latency, cold-start rate, wasted GB-seconds. *Controlled:* node hardware, k3s v1.29.4, OpenFaaS 0.27.x, images pre-pulled, cluster drained between runs, trace segment, seed set {1, 2, 3}.

ID	Name	Varies / answers	Output
E1	Memory sweep	policy × 6 ratios × 3 seeds — RQ1	Fig. 2
E2	Attribution	regress latency gap on image size and arrival CV — RQ2	Table 3
E3	Image stratification	small vs. large images at 25% — RQ3	Fig. 4

Cost model, run 2026-08-14. methods=2, ratios=6, seeds=3, replay=45 min ⇒ 2 × 6 × 3 = 36 runs × 0.75 h = 27 h, plus E2/E3 reusing E1 logs. Booked capacity: 10 h/week × 6 weeks = 60 h.

Fits, with slack. Adding a third policy would have pushed it to 40.5 h and left no slack — so the third policy was cut.

Reproducibility. Repo `github.com/<me>/faas-keepalive-study`; `requirements.txt` pinned; seeds recorded per run; `make figures` regenerates every plot from raw logs; node kernel, container runtime and image digests logged into each run directory.

Timeline. W1–2 cluster + pilot (crude end-to-end result by day 6); W3–5 reproduce published number; W6–9 E1; W10–12 E2/E3; W13–14 write-up.

YOUR WORK

Variables

Type	Variables
Independent (changed)	[...]
Dependent (measured)	[...]
Controlled (held fixed)	[hardware, versions, seeds, load, temperature]

Experiments

ID	Name	What it varies, and which RQ it answers	Output
E1	<i>[main comparison]</i>	<i>[...]</i>	<i>[Table 1]</i>
E2	<i>[ablation]</i>	<i>[...]</i>	<i>[Fig. 2]</i>
E3	<i>[sensitivity / stress]</i>	<i>[...]</i>	<i>[...]</i>

Cost model — run this before you commit

```

methods    = 3      # yours + how many baselines?
datasets   = 2
seeds      = 3      # minimum for any honest claim
ablations  = 4
hours_per_run = 1.5  # measured on ONE short pilot run, never guessed

runs = methods * datasets * seeds * (1 + ablations)
total = runs * hours_per_run
print(runs, "runs |", total, "compute-hours")

BUDGET = 60          # what you can actually book, not what exists
assert total <= BUDGET, f"need {total} h, have {BUDGET} -- narrow a knob"
```

My numbers. runs = [...], total = [...] hours, budget = [...] hours.

If the assertion fails, the knob I turned: [...]

Reproducibility

[Repository link, pinned dependency versions, seeds recorded, one command that regenerates every table and figure, hardware and runtime logged.]

Timeline

Weeks	Activity	Deliverable
<i>[1–2]</i>	<i>[data access + pilot run]</i>	<i>[crude end-to-end result]</i>
<i>[3–5]</i>	<i>[reproduce baseline]</i>	<i>[...]</i>
<i>[6–9]</i>	<i>[main experiments]</i>	<i>[...]</i>
<i>[10–12]</i>	<i>[ablations + repeats]</i>	<i>[...]</i>
<i>[13–14]</i>	<i>[write-up + figures]</i>	<i>[...]</i>

DONE WHEN

Variables classified into three lists. Every experiment maps to an RQ and names its output artefact. The cost model has been run with your own numbers and fits your budget. A crude end-to-end result exists, or is scheduled, in week one.

12 Evaluation Metrics

Lecture 1.4, 11.8 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Take every fuzzy word in your problem statement — “better”, “efficient”, “usable”, “fair” — and push it through the chain:

Construct → **Operational definition** → **Metric** → **Threshold vs. a named baseline**

Then interrogate each metric twice. The **validity gap**: does the metric still mean the construct? The **gaming gap**: can the metric be won without the construct? *Commits per week* measures activity, not value — and anyone who knows the metric can split one change into fifteen commits.

Report a **primary** metric (the claim rests on it), **secondary** metrics, and **guardrail** metrics that must not get worse.

COMMON TRAP

Metric shopping. Changing the metric or dataset until the result looks good, after seeing the results. This is a research-integrity violation, not a tuning step — and Unit III will name it. Fix the metric and threshold before the first run.

WORKED EXAMPLE — read it, then delete this whole box

Construct	Operational definition	Metric	Threshold
“responsive”	time from gateway receipt to first response byte, including queueing	p95 end-to-end latency (ms)	$\leq$ fixed-policy p95 – 20%
“efficient”	memory held by idle containers over the run	wasted GB-seconds	$\leq$ fixed policy
“stable”	run-to-run spread at the same configuration	SD of p95 across 3 seeds	$\leq$ 10% of mean

Primary: p95 end-to-end latency. **Secondary:** cold-start rate. **Guardrails:** wasted GB-seconds must not rise, and error rate must stay at 0.

Metric	Validity gap	Gaming gap
p95 latency	Excludes the requests that time out entirely, so a policy that drops the worst requests looks better	Raise the client timeout and the tail vanishes from the metric — so timeout is fixed at 30s and timeouts are counted separately
Cold-start rate	A cold start on a 95MB image is not the same event as one on a 1.1GB image	Route all traffic to the smallest image — so results are stratified by image

Statistics. 3 seeds minimum; p95 reported with bootstrap 95% CI (10000 resamples); Mann–Whitney U between policies at each ratio because latency is heavily skewed; Cliff’s delta as effect size; Holm correction across the six ratios.

Success criterion, fixed 2026-08-14 before the first measurement run. H1 is supported if and only if, at ratios $\leq 25\%$, the median-over-seeds p95 advantage of greedy-dual over fixed keep-alive is below 20% of the advantage measured at 100%, with non-overlapping bootstrap CIs.

YOUR WORK**Construct to threshold**

Construct	Operational definition	Metric	Threshold
[...]	[...]	[...]	[vs. named baseline]
[...]	[...]	[...]	[...]
[...]	[...]	[...]	[...]

Metric roles

Primary (the claim rests on this one): [...]

Secondary: [...]

Guardrails (must not get worse): [...]

Validity and gaming

Metric	Validity gap — where it stops meaning the construct	Gaming gap — how it could be won dishonestly
[...]	[...]	[...]
[...]	[...]	[...]

Statistical treatment

Seeds / repetitions: ≥ 3 **Reported as:** $[\text{mean} \pm SD, 95\% CI]$

Significance test and why it is appropriate: [...]

Effect size measure: [...]

Multiple-comparison correction, if testing many configurations: [...]

Success criterion, fixed in advance

“I will claim success if and only if ...” — write it before the first run, and do not edit it afterwards.

DONE WHEN

Every fuzzy word in the problem statement has a row. One primary metric is named. Each metric has both a validity gap and a gaming gap written down. The success criterion is fixed and dated before the first experiment.

13 Ethical Considerations

Lecture III.5–III.8 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Ethics is a gate, not a form — and you cannot narrow your way past it. Work through the five gates from Lecture I.4, then the integrity items from Unit III.

- G1. Human subjects or their data?** Consent, and ethics-committee approval.
- G2. Data obtained lawfully?** Licence, terms of service, `robots.txt`.
- G3. Could the result be misused?** Dual use; staged or withheld release.
- G4. Could it harm one group more than another?** Disaggregated evaluation.
- G5. Is it honestly reported?** Negative results, limitations, no metric shopping.

If your work touches no human data at all, say so explicitly and justify it — “not applicable” with a reason is a valid, and expected, answer.

COMMON TRAP

“It is public, so it is fine.” Visible is not consented, and public is not licensed. Scraping a site that forbids it in its terms of service is a violation whether or not anyone notices.

WORKED EXAMPLE — read it, then delete this whole box

	Gate	Applies	Answer / mitigation
G1	Human subjects or their data	No	The trace is anonymised function IDs and timestamps; no user, no content, no PII. Stated rather than assumed.
G2	Lawful acquisition	Yes	Trace is CC-BY-4.0 and cited. Cluster use is within the department’s acceptable-use policy; booking is on record.
G3	Misuse / dual use	Low	A keep-alive policy has no obvious offensive use. I will note that trace-replay tooling can be pointed at systems one does not own, and will not release a general load generator.
G4	Unequal harm	Indirect	None to people. I will report per-image results so the conclusion cannot be read as favouring large-image (well-resourced) deployments only.
G5	Honest reporting	Always	Success criterion fixed before the first run; the null result is the expected outcome and will be reported as the headline if it occurs.

Privacy and security. No personal data. Cluster credentials are in a local `.env`, git-ignored; the repo is scanned with `gitLeaks` before each push.

Bias and fairness. Not a human-facing model, so no group fairness analysis applies. The analogous concern is *benchmark bias*: the Azure trace is one provider’s, from 2019, and skews toward short functions. Stated as a limitation.

Integrity. Similarity check via Turnitin before submission. Sole author. **AI assistance:** used for LaTeX formatting and for drafting the plotting script; every number, citation and claim in the notebook was produced and checked by me, and I re-derived the statistics by hand for one configuration to confirm the script. Negative result committed to in advance.

YOUR WORK**The five gates**

	Gate	Applies?	My answer / mitigation
G1	Human subjects or their data	[Y/N]	[...]
G2	Lawful acquisition (licence, ToS)	[Y/N]	[...]
G3	Misuse / dual use	[Y/N]	[...]
G4	Unequal harm across groups	[Y/N]	[...]
G5	Honest reporting	Always	[...]

Privacy and security**Personal data involved:** *[what fields, whose]***Anonymisation / de-identification:** *[...]***Storage, access control and retention:** *[where it lives, who can read it, when it is deleted]***Bias and fairness****Groups I will report results separately for:** *[...]***Known bias in my data source:** *[...]***What I will do if a disparity appears:** *[...]***Research integrity**

- **Plagiarism.** *[quoting and paraphrasing policy; similarity check tool and date]*
- **Authorship and credit.** *[who contributed what]*
- **AI-assistance disclosure.** *[which tools, for what — writing, coding, analysis — and what you verified yourself]*
- **Negative results.** *[committed to reporting them]*
- **Conflicts of interest / funding.** *[...]*

Declaration

I confirm that the work recorded in this notebook is my own, that all sources are cited, and that any assistance is disclosed above.

Signature: _____

Date: _____

DONE WHEN

All five gates answered, including the ones marked not-applicable, each with a reason. Privacy, bias and integrity subsections are complete. The AI-assistance disclosure is specific about what was verified. The declaration is signed.

14 Research Proposal

Lecture IV.1–IV.5 Update this section after these lectures.

WHAT THIS SECTION IS FOR

The proposal is not new writing — it is Sections 1–13 assembled into a document someone outside the course could evaluate. If a subsection here needs material that is not already in your notebook, that is the notebook telling you what to work on next.

Target **8–12 pages** excluding references. Write for a reader who knows computer science but not your subfield.

COMMON TRAP

Relevance theatre. “This will help society and contribute to the nation’s growth.” A sentence that fits every project fits none. If your significance paragraph would survive being pasted into someone else’s proposal, it is not a significance paragraph.

WORKED EXAMPLE — read it, then delete this whole box

Title. *Does Greedy-Dual Keep-Alive Survive Memory Scarcity? A Replication and Attribution Study of Serverless Cold-Start Policies on a Constrained Cluster*

Abstract (excerpt). Serverless platforms reclaim idle containers, so rarely-invoked functions pay a cold start on almost every request. Greedy-dual keep-alive reduces cold starts roughly threefold on provider-scale infrastructure [Fuerst and Sharma \[2021\]](#), but published evaluations provision keep-alive memory generously relative to the function working set. Small on-premise clusters operate in the opposite regime. This work replays the public Azure Functions trace on a 3-node 8 GB cluster, sweeping the keep-alive memory ratio from 100% to 10%, and measures whether the policy’s p95 latency advantage over fixed 10-minute keep-alive survives. We further attribute any degradation to image size versus arrival burstiness. We hypothesise the advantage falls below 20% of its unconstrained value at ratios under 25%. If confirmed, small-cluster operators should retain the simpler fixed policy and invest instead in reducing image size . . .

*Sections 1–11 are assembled from Sections 1–13 of this notebook. The expected-contribution paragraph reads: “If the hypothesis holds, the contribution is *new evidence*: a documented boundary condition on a policy currently recommended without one, plus a released controller and replay harness that lets any operator measure their own ratio.”*

YOUR WORK

Title

[Specific enough that a reader knows the population, the task and the setting. Twelve to eighteen words.]

Abstract (150–250 words)

[Problem, gap, what you will do, how you will evaluate it, expected contribution. Write this last.]

1. Introduction and motivation

[From Sections 1 and 3. End with the “so what” chain from Section 3.]

2. Problem statement and research questions

[From Sections 3 and 4, verbatim.]

3. Literature review

[From Sections 6 and 7 — written as prose organised by concept, with the matrix as a table. Not a list of paper summaries.]

4. Research gap

[From Section 8.]

5. Objectives

[From Section 5.]

6. Proposed methodology

[From Section 9, including the pipeline figure and the baselines.]

7. Data and experimental plan

[From Sections 10 and 11, including the timeline.]

8. Evaluation

[From Section 12, including the success criterion fixed in advance.]

9. Ethical considerations

[From Section 13.]

10. Expected contribution and limitations

[What will be true that is not true today — in knowledge, practice, policy or cost. Then the limitations, stated by you before anyone else states them.]

11. Timeline and deliverables

[Gantt-style table or list.]

References

[Generated automatically from `references.bib`.]

Self-assessment before submission

Question	Yes/No
Could a stranger run my experiment from Section 7 alone?	<i>[...]</i>
Does every claim in Section 10 trace to a metric in Section 8?	<i>[...]</i>
Is there a result that would make me abandon my hypothesis, and is it stated?	<i>[...]</i>
Have I cited at least one paper that disagrees with me?	<i>[...]</i>
Does my significance paragraph fail the paste-into-someone-else's-proposal test?	<i>[...]</i>

DONE WHEN

Every subsection is drawn from an earlier notebook section, not written fresh. 8–12 pages. The self-assessment table is complete and honest. References compile from `references.bib`.

15 Final Presentation

Lecture IV.5 Update this section after these lectures.

WHAT THIS SECTION IS FOR

Record the **plan** for your defence and, afterwards, what actually happened. The second part is the one that is marked — it shows you listened.

Suggested arc for 10–12 slides: hook (the concrete failure or cost that motivates this) → problem statement → what is known → the gap → your question and hypothesis → method → data → evaluation plan → expected contribution → limitations and risks.

Rehearse the questions you hope nobody asks. They are the ones that get asked.

COMMON TRAP

Defending instead of thinking. If a question exposes a real weakness, say so, say what you would do about it, and move on. Examiners mark the quality of the answer, not the absence of the weakness.

WORKED EXAMPLE — read it, then delete this whole box

Slide 2, the hook. One number on the slide: *our lab cluster's p95 first-request latency for the ResNet-50 function is 4.2 s.* Then: the policy the literature recommends was measured where memory was never the binding constraint. Ours always is.

Two of the anticipated questions, answered.

“Why not just add more RAM?” — Because the claim is about the regime, not my cluster. Every edge and on-premise deployment lives in it, and the paper's recommendation carries no boundary condition. Adding RAM answers my inconvenience, not the question.

“What result would prove you wrong?” — If the advantage stays above 50% of the unconstrained gain all the way down to a 10% ratio, H1 is refuted and the policy generalises better than I claimed. I would report that as the headline.

After the defence (filled in on 2026-11-27). *Example of what this subsection should look like:* “I was asked why I used p95 rather than p99, and answered badly — I said ‘p99 is noisier’, which is true but is not a justification. The real reason is that at my invocation volume p99 rests on too few samples for a stable bootstrap CI. I have now added that sentence to Section 12.”

YOUR WORK

Slide plan

#	Slide	The one thing this slide must land	Min
1	Title	[...]	0.5
2	Hook	[a concrete failure, cost or number]	1
3	Problem statement	[...]	1
4	What is known	[...]	1.5
5	The gap	[...]	1
6	Question + hypothesis	[...]	1
7	Method	[...]	1.5
8	Data	[...]	1
9	Evaluation	[...]	1.5
10	Expected contribution	[...]	1
11	Limitations and risks	[...]	1

Anticipated questions — and my honest answers

Question I expect	My answer
Why this baseline and not a stronger one?	[...]
What result would prove you wrong?	[...]
Why is this novel — which of the four types?	[...]
Can you actually get this data?	[...]
Does your metric measure what you claim?	[...]
[the one you hope nobody asks]	[...]

After the defence — what actually happened

Date: [...]

Questions I was actually asked:

1. [...]
2. [...]
3. [...]

The question I answered worst, and what I now know: [...]

What I would change about the project: [...]

What I would change about how I worked: [...]

Closing reflection

Answer the question this notebook exists to answer, in one paragraph:

What is my research problem, why does it matter, what has already been done, what is still missing, how will I investigate it, how will I evaluate it, and what contribution do I expect to make?

[...]

DONE WHEN

Slide plan totals 10–12 minutes. Five or more anticipated questions have written answers. The post-defence section is filled in after the defence, including the question you answered worst. The closing reflection answers the course question in one paragraph.

References

- Alexandru Agache, Marc Brooker, Andreea Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *USENIX NSDI*, 2020.
- Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- Alexander Fuerst and Prateek Sharma. FaasCache: Keeping serverless computing alive with greedy-dual caching. In *ASPLOS*, 2021.
- Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. Serverless computing: One step forward, two steps back. In *Conference on Innovative Data Systems Research (CIDR)*, 2019.
- Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383*, 2019.
- Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *USENIX Annual Technical Conference (ATC)*, 2020.
- Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. Benchmarking, analysis, and optimization of serverless function snapshots. In *ASPLOS*, 2021.
- Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *EuroSys*, 2015.
- Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. Peeking behind the curtains of serverless platforms. In *USENIX Annual Technical Conference (ATC)*, 2018.